Digital Logic - Complete Notes

Coal India Limited - Management Trainee (Systems) | CBT Preparation

Header watermark: Asabhya Maanav

Table of Contents

Digital Logic - Orientation for CIL MT Systems CBT 5

1. Number Systems 6

1.1 Positional Notation and the Four Bases 6

1.2 Base Conversions (Any Base to Any Base) 6

1.3 Binary Arithmetic 7

1.4 Signed Number Representation 7

1.5 Range of n-bit Numbers 8

1.6 Subtraction Using Complements 8

1.7 Overflow Detection in Addition 9

2. Codes 10

2.1 BCD (8421) and Packed BCD 10

2.2 Excess-3 Code 10

2.3 Gray Code (and Binary <-> Gray Conversion) 10

2.4 ASCII / EBCDIC (Concept) 11

2.5 Error-Detecting Codes: Parity 11

2.6 Weighted vs Non-Weighted Codes 11

3. Boolean Algebra 13

3.1 Boolean Variables, Constants, Operations 13

3.2 Basic Laws: Identity, Null, Idempotent, Complement 13

3.3 Commutative, Associative, Distributive Laws 13

3.4 Absorption and Consensus Theorems 13

3.5 De Morgan's Theorems 14

3.6 Duality Principle 14

3.7 Boolean Function Representation: Truth Table and Expression 14

3.8 Canonical Forms: Minterms (SOP) and Maxterms (POS) 14

3.9 Conversion Between SOP and POS 15

3.10 Complement of a Function 15

4. Logic Minimization 16

4.1 Algebraic Simplification 16

4.2 Karnaugh Maps (K-map) 16

4.3 Quine-McCluskey Method (Tabular) - Concept 17

4.4 Hazards: Static and Dynamic (Concept) 17

5. Logic Gates 18

5.1 Basic Gates: AND, OR, NOT 18

5.2 Universal Gates: NAND, NOR 18

5.3 XOR and XNOR Gates 18

5.4 Gate-Level Implementation of Boolean Functions 19

5.5 Logic Families (TTL, CMOS) - Awareness 19

6. Combinational Circuits 20

6.1 Analysis and Design Procedure 20

6.2 Adders: Half Adder and Full Adder 20

6.3 Subtractors: Half and Full Subtractor 20

6.4 Ripple-Carry Adder; Carry Look-Ahead Adder (Concept) 20

6.5 Multiplexers (MUX) and Function Implementation 21

6.6 Demultiplexers (DEMUX) 21

6.7 Encoders (and Priority Encoder) 21

6.8 Decoders (and Function Implementation) 22

6.9 Magnitude Comparators 22

6.10 Code Converters (e.g., Binary-to-Gray) 22

6.11 Parity Generator/Checker 22

7. Latches and Flip-Flops 24

7.1 Latches: SR Latch, Gated SR, D Latch 24

7.2 Flip-Flops: SR, D, JK, T 24

7.3 Characteristic Tables and Excitation Tables 24

7.4 Flip-Flop Conversions (e.g., JK to D) 25

7.5 Edge-Triggered vs Level-Triggered; Master-Slave 25

7.6 Setup and Hold Time; Race-Around Condition 25

8. Registers and Counters 27

8.1 Registers; Shift Registers (SISO, SIPO, PISO, PIPO) 27

8.2 Universal Shift Register (Concept) 27

8.3 Asynchronous (Ripple) Counters 27

8.4 Synchronous Counters 27

8.5 Up/Down Counters; Mod-N Counters 28

8.6 Ring Counter and Johnson Counter 28

8.7 Counter Design from State Diagram (Concept) 28

9. Finite State Machines (Sequential Design) 30

9.1 Mealy Machine 30

9.2 Moore Machine 30
9.3 State Diagram and State Table 30
9.4 State Reduction/Minimization (Concept) 30
9.5 State Assignment (Concept) 30

10. Programmable & Memory Devices (Awareness) 32

10.1 ROM as Combinational Logic 32
10.2 PLA and PAL (Concept) 32
10.3 Multiplexer/Decoder-Based Implementation (Revisited) 32

11. Fixed and Floating Point 33

11.1 Fixed-Point Representation and Range 33
11.2 Floating-Point Representation; Normalization 33
11.3 IEEE 754: Single and Double Precision (Fields, Bias) 33
11.4 Overflow and Underflow 33
11.5 Floating-Point Addition/Multiplication (Concept) 34

12. Exam Focus / Practice Checklist 35

12.1 Base Conversions and Complement Arithmetic 35
12.2 K-map Minimization (SOP/POS, with don't-cares) 35
12.3 Counting Prime / Essential Prime Implicants 35
12.4 NAND/NOR Universal Realization 35
12.5 MUX/Decoder-Based Function Implementation 35
12.6 Flip-Flop Excitation and Conversion 35
12.7 Counter Mod/State-Sequence Questions 35
12.8 IEEE 754 Field/Bias Computation 35
12.9 Number Range for n-bit Representations 35

Digital Logic - Orientation for CIL MT Systems CBT

- Digital Logic is the foundation discipline that studies how **binary** information is represented, manipulated, and stored using **two** discrete voltage levels: logic **0** (LOW) and logic **1** (HIGH).
- The CIL MT Systems paper treats Digital Logic as a **high-yield, conceptual-to-moderate** area; emphasis falls on **number systems**, **Boolean simplification (K-maps)**, and **standard combinational/sequential building blocks**.
- A **digital** signal is discrete and noise-immune, whereas an **analog** signal is continuous; digital systems dominate because of **noise immunity**, **ease of design**, **storage**, and **programmability**.
- This guide proceeds in strict syllabus order: **Number Systems & Codes**, **Boolean Algebra**, **Logic Gates & Combinational Circuits**, **Sequential Circuits**, **Memory/Programmable Logic**, and **Computer Arithmetic (IEEE-754)**.

How to use this book

- Read each "##" section as a self-contained topic; revise the "#### CBT Trap" callouts the night before the exam.
- Every red token is an exam-answer candidate: a **number**, a **formula**, a **keyword**, or a **core fact**.

1. Number Systems

1.1 Positional Notation and the Four Bases

- A **positional number system** represents a value as a weighted sum of digits, where each position carries a weight equal to **base^position**.
- The general value formula for a number with integer digits d and fractional digits is **Value = Sigma (d_i x base^i)**, with i ranging from the most significant to least significant position.
- The **base** (or radix) is the count of distinct symbols a system uses; position weights are powers of the base.
- The four exam-relevant bases are **Binary (base 2)**, **Octal (base 8)**, **Decimal (base 10)**, and **Hexadecimal (base 16)**.

System	Base	Digit Symbols	Example
Binary	**2**	0,1	1011
Octal	**8**	0-7	753
Decimal	**10**	0-9	945
Hexadecimal	**16**	0-9, A-F	2AF

- In binary, each digit is a **bit**; **4 bits** form a **nibble** and **8 bits** form a **byte**.
- Hexadecimal digits A,B,C,D,E,F represent decimal **10,11,12,13,14,15** respectively.
- The **Most Significant Bit (MSB)** is the leftmost (highest weight); the **Least Significant Bit (LSB)** is the rightmost (weight **2^0 = 1**).

Memory Trick

- "BODH" ordering of bases by size: **Binary < Octal < Decimal < Hexadecimal** = **2 < 8 < 10 < 16**.
- Each hex digit maps to exactly **4** binary bits; each octal digit maps to exactly **3** binary bits.

CBT Trap

- A common trap: the largest single digit in base r is **r-1**, NOT r; in octal the digit **8** is illegal, in binary only **0 and 1** exist.
- Do not confuse **base** with **number of digits**; base 16 still uses single symbols per position (A-F), it is not "16 separate decimal columns".

1.2 Base Conversions (Any Base to Any Base)

- Conversion **from any base to decimal**: multiply each digit by its positional weight (base^position) and sum.
- Conversion **from decimal to any base (integer part)**: use **repeated division** by the target base; the remainders read **bottom-to-top** give the result.
- Conversion **from decimal to any base (fractional part)**: use **repeated multiplication** by the target base; the integer parts read **top-to-bottom** give the result.
- **Binary to Octal**: group bits in **3s** from the radix point; **Binary to Hex**: group bits in **4s** from the radix point.
- **Octal to Binary**: expand each octal digit to **3 bits**; **Hex to Binary**: expand each hex digit to **4 bits**.
- **Octal to Hex** (or vice versa): the safest path is **via binary** (octal -> binary -> regroup into 4s -> hex).

Conversion	Method	Read order
Any base -> Decimal	Multiply by weights, sum	n/a
Decimal -> Any (integer)	**Repeated division**, keep remainders	**bottom to top**
Decimal -> Any (fraction)	**Repeated multiplication**, keep carries	**top to bottom**
Binary -> Octal	group **3 bits**	from radix point

Conversion	Method	Read order
Binary -> Hex	group **4 bits**	from radix point

- Worked example: **(1011.101)_2 = 8+0+2+1 + 0.5+0+0.125 = (11.625)_10**.
- Worked example: **(25)_10 = 11001_2** via 25/2=12r1, 12/2=6r0, 6/2=3r0, 3/2=1r1, 1/2=0r1.
- Worked example: **(0.625)_10 = 0.101_2** via 0.625x2=1.25(1), 0.25x2=0.5(0), 0.5x2=1.0(1).

Memory Trick

- "Divide-Down for integers, Multiply-Up for fractions" -> remainders go **up**, carries go **down**.

CBT Trap

- When grouping binary for octal/hex, pad with **leading zeros** for the integer part and **trailing zeros** for the fractional part - padding direction is opposite for the two sides of the radix point.
- Fractional decimal-to-binary may **never terminate** (e.g., 0.1 decimal is non-terminating in binary); stop at the requested precision.

1.3 Binary Arithmetic

- Binary addition rules: **0+0=0**, **0+1=1**, **1+0=1**, **1+1=10** (sum 0, carry 1), and **1+1+1=11** (sum 1, carry 1).
- Binary subtraction rules: **0-0=0**, **1-0=1**, **1-1=0**, **0-1=1 with borrow 1**.
- Binary multiplication is a series of **shift-and-add** operations; multiplying by **2^n** shifts left by n positions.
- Binary division uses **shift-and-subtract** (long division); dividing by **2^n** shifts right by n positions.
- Multiplication partial products use rules **0x0=0, 0x1=0, 1x0=0, 1x1=1**.

Operation	Key rule	Shortcut
Addition	**1+1 = 0 carry 1**	half/full adder
Subtraction	**0-1 = 1 borrow 1**	use 2's complement
Multiply by 2^n	**left shift n**	append n zeros
Divide by 2^n	**right shift n**	drop n LSBs

- Worked example: **1011 + 0110 = 10001** (11+6 = 17).
- Worked example: **1101 x 101 = 1000001** (13x5 = 65).

CBT Trap

- The biggest error is mishandling the **carry chain**; in 1+1+1 the result is **sum 1, carry 1**, not "sum 0".
- For subtraction, examiners prefer you **convert to 2's complement addition** rather than borrow-by-hand.

1.4 Signed Number Representation

- There are **3** standard schemes to represent signed integers: **Sign-Magnitude**, **1's complement**, and **2's complement**.
- In all three, the **MSB** acts as the **sign bit**: **0 = positive**, **1 = negative**.

1.4.1 Sign-Magnitude

- The MSB is the sign; the remaining **n-1** bits store the **magnitude** in plain binary.
- Sign-magnitude has **two** representations of zero: **+0 = 0000** and **-0 = 1000**.
- Range for n bits is **-(2^(n-1) - 1)** to **+(2^(n-1) - 1)**.
- Disadvantage: arithmetic requires **separate sign handling**; rarely used for integer ALUs.

1.4.2 1's Complement

- The 1's complement of a number is obtained by **inverting every bit** (bitwise NOT).
- It also has **two** zeros: **+0 = 0000** and **-0 = 1111**.
- Range for n bits is **-(2^(n-1) - 1)** to **+(2^(n-1) - 1)**.
- Addition needs an **end-around carry**: any carry out of the MSB is **added back** to the LSB.

1.4.3 2's Complement

- The 2's complement is obtained by **inverting all bits and adding 1** (i.e., 1's complement + 1).
- Shortcut: copy bits from LSB up to and including the **first 1**, then **invert** the rest.
- It has a **single, unique** representation of zero (**0000**).
- Range for n bits is **-(2^(n-1))** to **+(2^(n-1) - 1)** - asymmetric, one extra negative value.
- It is the **universal standard** for modern computers because addition/subtraction use the **same** adder hardware and carry-out is simply **discarded**.

Scheme	Zero reps	n-bit range	Carry handling
Sign-Magnitude	**two (+0,-0)**	**-(2^(n-1)-1) to +(2^(n-1)-1)**	special
1's complement	**two (+0,-0)**	**-(2^(n-1)-1) to +(2^(n-1)-1)**	**end-around carry**
2's complement	**one**	**-2^(n-1) to +(2^(n-1)-1)**	**discard carry**

Memory Trick

- "2's complement = 1's complement + 1"; the lone **negative extra** value belongs to 2's complement (e.g., **-8** in 4 bits).

CBT Trap

- Only **2's complement** has a unique zero and the asymmetric range; the other two have **dual zeros** and a symmetric range.
- For an 8-bit value, **-128** is representable in 2's complement but **+128** is NOT.

1.5 Range of n-bit Numbers

- Unsigned n-bit range: **0** to **2^n - 1**, giving **2^n** distinct values.
- Signed 2's complement n-bit range: **-2^(n-1)** to **+2^(n-1) - 1**.
- Number of representable values is always **2^n** regardless of signed/unsigned interpretation.

n bits	Unsigned max	2's comp min	2's comp max
4	**15**	**-8**	**+7**
8	**255**	**-128**	**+127**
16	**65535**	**-32768**	**+32767**
32	**4294967295**	**-2147483648**	**+2147483647**

CBT Trap

- Off-by-one: unsigned max is **2^n - 1** (subtract one), while the **count** of values is **2^n**.

1.6 Subtraction Using Complements

- To compute A - B in 2's complement: form the **2's complement of B** and **add** it to A.
- If a **carry-out** of the MSB is produced, the result is **positive** and the carry is **discarded**.
- If **no carry-out** occurs, the result is **negative** and is itself in **2's complement** form (take its 2's complement to read the magnitude).
- In 1's complement subtraction, a carry-out triggers an **end-around carry** (add 1 to LSB).

- Worked example (4-bit): **7 - 5**: 2's comp of 5 = 1011; 0111 + 1011 = 1 0010 -> discard carry -> **0010 = +2**.

CBT Trap

- The **carry-out meaning differs** between 1's complement (add it back) and 2's complement (discard it) - mixing them is the classic trap.

1.7 Overflow Detection in Addition

- **Overflow** occurs only when **two operands of the same sign** produce a result of the **opposite sign**.
- Adding numbers of **opposite signs** can **never** overflow.
- Rule 1: overflow if **sign(A) = sign(B)** but **sign(result) != sign(A)**.
- Rule 2 (hardware): overflow = **carry-into-MSB XOR carry-out-of-MSB** = **C_in(MSB) XOR C_out(MSB)**.
- Worked example (4-bit): **0111 (+7) + 0001 (+1) = 1000 (-8)** -> sign flipped -> **overflow**.

Memory Trick

- "Same signs in, different sign out = overflow"; for hardware, "the two top carries disagree".

CBT Trap

- A **carry-out** is NOT the same as **overflow**; in 2's complement a carry-out is often discarded normally, while overflow is the **XOR of the two MSB carries**.

2. Codes

2.1 BCD (8421) and Packed BCD

- **Binary Coded Decimal (BCD)** encodes each decimal digit (0-9) in its own **4-bit** group using weights **8-4-2-1**.
- BCD is a **weighted** code; the combinations **1010 to 1111** (decimal 10-15) are **invalid/illegal** in BCD.
- **Packed BCD** stores **two** decimal digits in one byte (**4 bits each**); unpacked BCD uses one byte per digit.
- BCD addition needs a correction: if a nibble sum exceeds **9** or generates a carry, **add 6 (0110)**.
- Example: decimal **59** in BCD = **0101 1001**, which differs from pure binary **111011**.

Feature	Pure Binary	BCD (8421)
Digit encoding	whole number	**per decimal digit**
Weights	2^i	**8,4,2,1**
Invalid codes	none	**1010-1111**
Storage efficiency	**high**	**lower**

CBT Trap

- **(59)_10** in BCD is **0101 1001**, NOT the pure binary **111011**; mixing the two is a frequent trap.
- BCD wastes codes (only 10 of 16 used), so it is **less storage-efficient** than binary.

2.2 Excess-3 Code

- **Excess-3 (XS-3)** is obtained by **adding 3 (0011)** to the BCD value of each digit.
- Excess-3 is a **non-weighted** and **self-complementing** code: the **9's complement** equals its **1's complement**.
- Example: decimal **5** = BCD 0101 -> Excess-3 = **1000** (5+3=8).
- Self-complementing property eases **BCD subtraction** hardware.

Decimal	BCD (8421)	Excess-3
0	0000	**0011**
4	0100	**0111**
5	0101	**1000**
9	1001	**1100**

CBT Trap

- Excess-3 is **non-weighted** yet **self-complementing**; do not call it weighted just because it derives from 8421 BCD.

2.3 Gray Code (and Binary <-> Gray Conversion)

- **Gray code** is a **non-weighted**, **reflected**, **cyclic** code where **exactly one bit** changes between consecutive values (**unit distance / minimum-change** code).
- Binary -> Gray: the MSB is **copied**; each subsequent Gray bit = **binary_bit XOR previous_binary_bit**.
- Gray -> Binary: the MSB is **copied**; each subsequent binary bit = **Gray_bit XOR previous_binary_bit** (running XOR).
- Gray code minimizes **switching errors** in K-maps, shaft encoders, and asynchronous transitions.
- Example: binary **1011** -> Gray **1110**; Gray **1110** -> binary **1011**.

Decimal	Binary	Gray
0	000	**000**
1	001	**001**
2	010	**011**
3	011	**010**
4	100	**110**

Memory Trick

- Binary->Gray uses **XOR of adjacent binary bits**; Gray->Binary uses a **running (cumulative) XOR**.

CBT Trap

- Direction matters: Binary->Gray XORs **the original binary bits**; Gray->Binary XORs **the already-computed binary bit** - swapping them gives wrong answers.
- Gray code is **non-weighted**; you cannot assign positional weights to it.

2.4 ASCII / EBCDIC (Concept)

- **ASCII** (American Standard Code for Information Interchange) is a **7-bit** character code representing **128** symbols; extended ASCII uses **8 bits** for **256**.
- ASCII 'A' = **65 (0x41)**, 'a' = **97 (0x61)**, '0' = **48 (0x30)**; difference between a letter's upper and lower case is **32**.
- **EBCDIC** (Extended Binary Coded Decimal Interchange Code) is an **8-bit** IBM mainframe code representing **256** symbols.
- ASCII is the dominant standard on PCs and the internet; EBCDIC survives on legacy **IBM mainframes**.

Code	Bits	Symbols	Used by
ASCII	**7**	**128**	PCs, internet
Extended ASCII	**8**	**256**	PCs
EBCDIC	**8**	**256**	**IBM mainframes**

CBT Trap

- 'a' - 'A' = **32**; ASCII '0' is **48** not 0 - converting a digit char to value needs subtracting **48**.

2.5 Error-Detecting Codes: Parity

- **Parity** adds one **parity bit** to make the total number of 1s **even (even parity)** or **odd (odd parity)**.
- A single parity bit detects **all odd numbers of bit errors** (notably **single-bit** errors) but **cannot correct** any.
- Parity cannot detect an **even number** of bit errors (e.g., **2** simultaneous flips).
- **Hamming code** extends this idea with multiple parity bits to **detect and correct** single-bit errors.

Scheme	Detects	Corrects
Single parity	**odd-count errors**	**none**
2-D parity	**many errors**	**single-bit**
Hamming (SEC)	**2-bit**	**1-bit**

CBT Trap

- Parity **detects but never corrects**; it misses **even-numbered** errors entirely.

2.6 Weighted vs Non-Weighted Codes

- A **weighted** code assigns a fixed positional weight to each bit (e.g., **8421 BCD**, **2421**, **84-2-1**).

- A **non-weighted** code has no positional weights (e.g., **Excess-3**, **Gray**).
- **Self-complementing** weighted codes include **2421** and **84-2-1** (weights sum to **9**); Excess-3 is self-complementing but non-weighted.
- **Sequential** codes have each successive code one binary number higher than the previous (e.g., **8421**, **Excess-3**).

Code	Weighted?	Self-complementing?
8421 BCD	**Yes**	**No**
2421	**Yes**	**Yes**
Excess-3	**No**	**Yes**
Gray	**No**	**No**

Memory Trick

- Self-complementing codes have digit weights summing to **9** (2+4+2+1 = 9).

CBT Trap

- **8421 BCD** is weighted but **NOT** self-complementing; **2421** is both - examiners swap these.

3. Boolean Algebra

3.1 Boolean Variables, Constants, Operations

- **Boolean algebra** (formulated by **George Boole**, applied to switching circuits by **Claude Shannon**, 1938) operates on variables that take only **two** values: **0** and **1**.
- The **three** fundamental operations are **AND (.)** (logical product), **OR (+)** (logical sum), and **NOT (')** (complement/inversion).
- Constants are **0** and **1**; a Boolean variable plus its operations form a **switching algebra**.
- Operator precedence is **NOT > AND > OR** (complement first, then product, then sum).

3.2 Basic Laws: Identity, Null, Idempotent, Complement

- **Identity law**: **A + 0 = A** and **A . 1 = A**.
- **Null (dominance) law**: **A + 1 = 1** and **A . 0 = 0**.
- **Idempotent law**: **A + A = A** and **A . A = A**.
- **Complement law**: **A + A' = 1** and **A . A' = 0**.
- **Involution (double negation)**: **(A')' = A**.

Law	OR form	AND form
Identity	**A+0=A**	**A.1=A**
Null	**A+1=1**	**A.0=0**
Idempotent	**A+A=A**	**A.A=A**
Complement	**A+A'=1**	**A.A'=0**

CBT Trap

- **A + 1 = 1** (null) but **A . 1 = A** (identity); the constant 1 behaves oppositely under OR vs AND.

3.3 Commutative, Associative, Distributive Laws

- **Commutative**: **A + B = B + A**, **A . B = B . A**.
- **Associative**: **(A+B)+C = A+(B+C)**, **(A.B).C = A.(B.C)**.
- **Distributive**: **A.(B+C) = A.B + A.C** and the dual **A + B.C = (A+B).(A+C)**.
- The second distributive form (**OR distributes over AND**) is **unique to Boolean algebra** and has no ordinary-arithmetic analogue.

CBT Trap

- **A + B.C = (A+B).(A+C)** is valid in Boolean algebra but false in ordinary algebra - a favourite distractor.

3.4 Absorption and Consensus Theorems

- **Absorption law**: **A + A.B = A** and **A.(A+B) = A**.
- **Absorption variant**: **A + A'.B = A + B** and **A.(A'+B) = A.B**.
- **Consensus theorem**: **A.B + A'.C + B.C = A.B + A'.C** - the **B.C** term is **redundant**.
- **Dual consensus**: **(A+B).(A'+C).(B+C) = (A+B).(A'+C)**.

Memory Trick

- In consensus, the redundant term contains the **two variables** that appear **un-complemented and complemented** in the other two terms.

CBT Trap

- The consensus (redundant) term is the one formed from the **opposing literals**; students wrongly delete a needed term - verify with a truth table if unsure.

3.5 De Morgan's Theorems

- **De Morgan's Theorem 1**: **(A.B)' = A' + B'** - complement of a product is the sum of complements.
- **De Morgan's Theorem 2**: **(A+B)' = A'.B'** - complement of a sum is the product of complements.
- Generalization: to complement a function, **complement each variable** and **swap AND with OR** (and **0 with 1**).
- De Morgan's laws underlie **NAND/NOR universality**.

Memory Trick

- "Break the bar, change the sign": split the long bar and flip **AND<->OR**.

CBT Trap

- You must **change the operator** when applying De Morgan; **(A.B)' is NOT A'.B'** - that is the single most common Boolean error.

3.6 Duality Principle

- The **duality principle** states that any valid Boolean identity remains valid if you **swap AND<->OR** and **swap 0<->1**, leaving variables unchanged.
- Dual of **A + 0 = A** is **A . 1 = A**; dual of **A + 1 = 1** is **A . 0 = 0**.
- Duality differs from **complement**: duality does **not** invert the variables, whereas De Morgan's complement **does**.

CBT Trap

- **Dual** keeps variables as-is; **complement (De Morgan)** negates variables too - distinguishing them is a guaranteed MCQ.

3.7 Boolean Function Representation: Truth Table and Expression

- A Boolean function of n variables has a **truth table** with **2^n** rows enumerating every input combination.
- The function can be written as an **algebraic expression**, drawn as a **logic circuit**, or listed as **minterms/maxterms**.
- A function of n variables has **2^(2^n)** possible distinct functions (e.g., **16** functions of 2 variables).

CBT Trap

- Rows = **2^n**; total distinct functions = **2^(2^n)** - do not confuse the two exponentials.

3.8 Canonical Forms: Minterms (SOP) and Maxterms (POS)

- A **minterm** is a product (AND) term containing **all** variables, true for **exactly one** input combination; denoted **m_i**.
- A **maxterm** is a sum (OR) term containing **all** variables, false for **exactly one** combination; denoted **M_i**.
- **Canonical SOP** (Sum of Products) = **OR of minterms** where the function is **1**: F = Sigma m(...).
- **Canonical POS** (Product of Sums) = **AND of maxterms** where the function is **0**: F = Pi M(...).

- In a minterm a variable appears **complemented if its bit is 0**; in a maxterm a variable appears **complemented if its bit is 1** (opposite convention).
- Relationship: **m_i = (M_i)'** and **M_i = (m_i)'**.

Form	Built from	Term type	Index where
Canonical SOP	**minterms**	**AND terms ORed**	**F = 1**
Canonical POS	**maxterms**	**OR terms ANDed**	**F = 0**

Memory Trick

- "SOP -> sum of minterms where output is 1"; "POS -> product of maxterms where output is 0".

CBT Trap

- Minterm vs maxterm complement convention is **reversed**: in minterms 0->complemented, in maxterms 1->complemented.

3.9 Conversion Between SOP and POS

- The minterm and maxterm index sets are **complementary**: minterm indices (F=1) and maxterm indices (F=0) together cover **all 2^n** indices with **no overlap**.
- To convert **Sigma m(list)** to POS: take **Pi M** of the **missing** indices.
- To convert **Pi M(list)** to SOP: take **Sigma m** of the **missing** indices.
- Example (3-var): F = Sigma m(0,2,5,7) -> F = **Pi M(1,3,4,6)**.

CBT Trap

- The conversion uses the **complementary index set**, not the same indices; also F' uses the **remaining** minterms.

3.10 Complement of a Function

- The complement **F'** is found by applying **De Morgan's theorem** to the whole expression, or by **taking the dual and complementing each literal**.
- In canonical form: if F = Sigma m(set), then **F' = Sigma m(complementary set)**.
- Equivalently if F = Sigma m(A), then **F' = Pi M(A)** over the same index set.

CBT Trap

- **F' = Sigma m** of the **other** minterms = **Pi M** of the **same** minterm indices - both views are equivalent but mixing the index sets is the trap.

4. Logic Minimization

4.1 Algebraic Simplification

- **Algebraic simplification** repeatedly applies Boolean laws (absorption, consensus, distribution, De Morgan) to reduce literal/term count.
- Goal: fewer **literals** and **gates**, lowering cost and **propagation delay**.
- Drawback: there is **no systematic guarantee** of reaching the minimum - it depends on insight; hence **K-maps** and **Quine-McCluskey** are preferred.

CBT Trap

- Algebraic methods are **not guaranteed minimal**; the exam expects you to know K-map/Q-M give the **systematic** minimum.

4.2 Karnaugh Maps (K-map)

- A **K-map** is a graphical grid of **2^n** cells (one per minterm) arranged so adjacent cells differ by **one variable (Gray-code ordering)**.
- Minimization works by **grouping adjacent 1s** (for SOP) into the **largest power-of-2** groups.

4.2.1 2, 3, 4-variable K-maps

- A 2-variable map has **4** cells, 3-variable has **8** cells, 4-variable has **16** cells.
- Row/column labels follow **Gray code (00,01,11,10)** so physically adjacent cells are logically adjacent.
- The map **wraps around**: leftmost/rightmost columns and top/bottom rows are adjacent (toroidal).

4.2.2 Grouping rules; prime implicants

- Groups must contain **1, 2, 4, 8, ... (powers of 2)** cells and be **rectangular**.
- Each group of **2^k** cells eliminates **k** variables from that product term.
- A **prime implicant (PI)** is a group that **cannot be enlarged** (combined further) into a bigger valid group.
- Always make groups **as large as possible** and **as few as possible**.

4.2.3 Essential prime implicants

- An **essential prime implicant (EPI)** is a prime implicant that covers at least **one minterm not covered by any other PI**.
- The minimal cover must include **all EPIs**; remaining uncovered minterms are filled by selecting additional PIs.

4.2.4 Don't-care conditions

- A **don't-care** (denoted **X / d / phi**) is an input combination that **never occurs** or whose output **does not matter**.
- Don't-cares may be treated as **0 or 1** - whichever yields **larger groups** and simpler logic.
- They appear in K-maps and in the notation **F = Sigma m(...) + d(...)**.

4.2.5 SOP and POS minimization

- For **SOP**, group the **1s** (and useful don't-cares); each group gives a **product term**.
- For **POS**, group the **0s** (and useful don't-cares); each group gives a **sum term** (read complemented variables).

Concept	Definition	Exam keyword
Implicant	any valid group of 1s	product term
Prime implicant	**cannot be made larger**	PI

Concept	Definition	Exam keyword
Essential PI	**covers a 1 no other PI covers**	EPI
Don't-care	output **irrelevant**	X

Memory Trick

- "Biggest groups, fewest groups, powers of 2, wrap around the edges."

CBT Trap

- A group of **2^k** removes exactly **k** variables; counting PIs vs EPIs is a classic CIL question - **all EPIs are PIs, but not all PIs are essential**.
- For POS you group **0s**, not 1s - forgetting to switch is the trap.

4.3 Quine-McCluskey Method (Tabular) - Concept

- **Quine-McCluskey (Q-M)** is a **tabular, algorithmic** minimization method suitable for **many variables** and **computer implementation** (unlike the visual K-map limited to ~**6** variables).
- Step 1: list minterms grouped by their **number of 1s**.
- Step 2: **combine** terms differing in exactly **one bit** (mark the differing bit with a dash) to find **prime implicants**.
- Step 3: build a **prime implicant chart** to select **essential PIs** and a minimal cover.
- Q-M and K-map yield the **same** minimal result; Q-M is more **systematic but tedious** by hand.

CBT Trap

- Q-M finds **all prime implicants** systematically; its complexity grows **exponentially** in the worst case, but it is **automatable** whereas K-maps are visual only.

4.4 Hazards: Static and Dynamic (Concept)

- A **hazard** is an unwanted, momentary **glitch** in output caused by unequal **propagation delays** along different paths.
- **Static-1 hazard**: output should stay **1** but momentarily dips to **0**.
- **Static-0 hazard**: output should stay **0** but momentarily rises to **1**.
- **Dynamic hazard**: output makes **multiple transitions** (e.g., 1->0->1->0) when it should change just once.
- Cure: add **redundant (consensus) terms** to overlap adjacent K-map groups, removing the static hazard.

Hazard	Symptom	Fix
Static-1	**1->0->1 glitch**	**redundant SOP term**
Static-0	**0->1->0 glitch**	redundant POS term
Dynamic	**multiple transitions**	hazard-free design

CBT Trap

- Static hazards are removed by **adding redundant (consensus) terms** - the same terms minimization tries to delete; static-1 relates to **SOP**, static-0 to **POS**.

5. Logic Gates

5.1 Basic Gates: AND, OR, NOT

- The **AND** gate outputs **1** only when **all** inputs are 1; expression **Y = A.B**.
- The **OR** gate outputs **1** when **at least one** input is 1; expression **Y = A+B**.
- The **NOT** (inverter) outputs the complement; expression **Y = A'**.
- These three gates form a **functionally complete** set on their own.

Gate	Expression	Output 1 when
AND	**A.B**	**all inputs 1**
OR	**A+B**	**any input 1**
NOT	**A'**	**input is 0**

5.2 Universal Gates: NAND, NOR

- **NAND** = **(A.B)'** and **NOR** = **(A+B)'**; each is called a **universal gate** because **any** Boolean function can be built using **only NAND** or **only NOR**.
- NOT from NAND: tie inputs together -> **(A.A)' = A'**.
- AND from NAND: **NAND followed by NAND-inverter** (two NANDs).
- OR from NAND: **invert both inputs then NAND** (three NANDs) by De Morgan.
- A 2-input function may need **more** gates in pure-NAND form but uses only **one gate type** (cheaper fabrication).

To build	Using NAND	Using NOR
NOT	**1 NAND**	**1 NOR**
AND	**2 NAND**	**3 NOR**
OR	**3 NAND**	**2 NOR**

Memory Trick

- NAND builds **AND/OR** symmetrically; "NAND is the carpenter of AND, NOR is the carpenter of OR" (each builds its same-name function with fewest gates).

CBT Trap

- Both **NAND** and **NOR** are universal; **XOR** is **NOT** universal. Building OR from NAND needs **3** gates, not 2.

5.3 XOR and XNOR Gates

- **XOR** (exclusive-OR) outputs **1** when inputs **differ**; **Y = A XOR B = A'B + AB'**.
- **XNOR** (equivalence) outputs **1** when inputs **are equal**; **Y = (A XOR B)' = AB + A'B'**.
- XOR of n bits = **1** if the count of 1s is **odd** (basis of **parity** generation).
- Useful identities: **A XOR 0 = A**, **A XOR 1 = A'**, **A XOR A = 0**, **A XOR A' = 1**.

Gate	Output 1 when	Expression
XOR	**inputs differ**	**A'B + AB'**
XNOR	**inputs equal**	**AB + A'B'**

CBT Trap

- **XNOR** = equality detector; for **multi-bit**, even number of XNOR inversions flips meaning - a 3-input XNOR is **not** simply "all equal".

5.4 Gate-Level Implementation of Boolean Functions

- Any SOP expression maps to a **two-level AND-OR** network; any POS maps to a **two-level OR-AND** network.
- **AND-OR** converts directly to an all-**NAND** circuit; **OR-AND** converts directly to an all-**NOR** circuit (by inserting paired inversions).
- **Fan-in** = number of inputs a gate accepts; **fan-out** = number of gate inputs one output can drive.

CBT Trap

- A two-level **SOP** maps cleanly to **NAND-NAND**; a two-level **POS** maps to **NOR-NOR** - swapping these is a frequent error.

5.5 Logic Families (TTL, CMOS) - Awareness

- **TTL** (Transistor-Transistor Logic) uses **BJTs**; operates at **5V**, faster historically but higher **power**.
- **CMOS** (Complementary MOS) uses **MOSFETs**; very **low static power**, high **noise margin**, high **packing density** - dominant today.
- **ECL** (Emitter-Coupled Logic) is the **fastest** but most **power-hungry**.
- **Fan-out** of standard TTL is about **10**.

Family	Devices	Power	Speed
TTL	**BJT**	medium	medium
CMOS	**MOSFET**	**very low**	high
ECL	BJT	**high**	**fastest**

CBT Trap

- **CMOS** has the lowest **static** power; **ECL** is fastest. Do not say TTL is lowest power.

6. Combinational Circuits

6.1 Analysis and Design Procedure

- A **combinational circuit** output depends **only on present inputs** (no memory, no feedback).
- Design steps: **(1)** state the problem, **(2)** assign input/output variables, **(3)** build the **truth table**, **(4)** derive simplified expressions (K-map), **(5)** draw the **logic diagram**.
- Analysis is the reverse: from circuit -> expression -> truth table -> function.

CBT Trap

- Combinational = **no memory / no feedback**; the moment feedback or a clock appears it becomes **sequential**.

6.2 Adders: Half Adder and Full Adder

- A **half adder** adds **two** bits: **Sum = A XOR B**, **Carry = A.B**; it has **no carry-in**.
- A **full adder** adds **three** bits (A, B, Cin): **Sum = A XOR B XOR Cin**, **Cout = AB + Cin(A XOR B)**.
- A full adder can be built from **two half adders + one OR** gate.
- n-bit addition cascades **n** full adders.

Circuit	Inputs	Sum	Carry
Half adder	**2**	**A XOR B**	**A.B**
Full adder	**3**	**A XOR B XOR Cin**	**AB+BCin+ACin**

Memory Trick

- "Sum = XOR of all bits; Carry = majority (any two) of the bits."

CBT Trap

- Full-adder carry is the **majority function AB+BCin+ACin**, equivalently **AB + Cin(A XOR B)** - not simply A.B.

6.3 Subtractors: Half and Full Subtractor

- A **half subtractor** computes A-B: **Difference = A XOR B**, **Borrow = A'.B**.
- A **full subtractor** includes borrow-in: **Difference = A XOR B XOR Bin**, **Borrow = A'B + A'Bin + B.Bin**.
- Subtraction is usually done by **2's complement addition**, reusing the adder hardware.

Circuit	Difference	Borrow
Half subtractor	**A XOR B**	**A'.B**
Full subtractor	**A XOR B XOR Bin**	**A'B + A'Bin + B.Bin**

CBT Trap

- Half-subtractor borrow is **A'.B** (note the complement on A), whereas half-adder carry is **A.B** - the difference of one complement flips the answer.

6.4 Ripple-Carry Adder; Carry Look-Ahead Adder (Concept)

- A **Ripple-Carry Adder (RCA)** chains n full adders; the carry must **propagate** through all stages, giving worst-case delay **O(n)** (proportional to n).
- A **Carry Look-Ahead Adder (CLA)** computes carries in parallel using **Generate G_i = A_i.B_i** and **Propagate P_i = A_i XOR B_i**, reducing delay to about **O(log n)**.

- Carry equation: **C_(i+1) = G_i + P_i.C_i**.
- CLA is **faster but more hardware**; RCA is **simpler but slow**.

Adder	Carry delay	Hardware
Ripple-carry	**O(n)**	**minimal**
Carry look-ahead	**O(log n)**	**more gates**

Memory Trick

- "Generate when both 1 (G=A.B); Propagate when they differ (P=A XOR B)."

CBT Trap

- **Generate** = **A.B**, **Propagate** = **A XOR B** (some texts use A+B for propagate in carry-out but XOR for sum) - the exam answer for delay reduction is the **parallel carry computation**.

6.5 Multiplexers (MUX) and Function Implementation

- A **multiplexer (MUX)** is a **many-to-one** data selector: **2^n** data inputs, **n** select lines, **1** output.
- The output equals the data input addressed by the select lines: a **4:1 MUX** needs **2** select lines.
- A MUX is a **universal** building block: an n-select MUX implements **any function of n variables** directly, and any function of **n+1** variables using one variable at the data inputs.
- To implement an n-variable function with a **2^(n-1):1** MUX, apply (n-1) variables to selects and feed **0, 1, var, var'** to the data inputs.

MUX size	Data inputs	Select lines
2:1	**2**	**1**
4:1	**4**	**2**
8:1	**8**	**3**
2^n:1	**2^n**	**n**

Memory Trick

- "MUX = Many inputs, one output, selected by n lines for 2^n inputs."

CBT Trap

- A **2^n:1** MUX has **n** select lines (log relationship); a **4:1** needs **2** selects, not 4 - confusing data count with select count is the trap.

6.6 Demultiplexers (DEMUX)

- A **demultiplexer (DEMUX)** is the reverse: **one** input routed to **2^n** outputs using **n** select lines (a **one-to-many** distributor).
- A DEMUX with its input tied to **1** behaves exactly like a **decoder**.

CBT Trap

- A **DEMUX with input = 1** is functionally a **decoder**; MUX and DEMUX are inverses - selecting the wrong direction is the trap.

6.7 Encoders (and Priority Encoder)

- An **encoder** converts **2^n** input lines into an **n-bit** code; only one input is assumed **active** at a time.
- A plain encoder is ambiguous when **multiple inputs** are active or when **no input** is active.

- A **priority encoder** resolves multiple active inputs by encoding the **highest-priority (usually highest-numbered)** active input.
- An **8-to-3** encoder has **8** inputs and **3** outputs.

Device	Inputs	Outputs
8-to-3 encoder	**8**	**3**
Decimal-to-BCD	**10**	**4**
Priority encoder	**2^n**	**n + valid bit**

CBT Trap

- A plain encoder fails on **simultaneous active inputs**; the **priority encoder** fixes this by ranking inputs - the exam asks which handles multiple highs.

6.8 Decoders (and Function Implementation)

- A **decoder** converts an **n-bit** code into **2^n** output lines, activating **exactly one** (each output = one minterm).
- An **n-to-2^n** decoder plus an **OR** gate implements **any SOP function** (OR the relevant minterm outputs).
- A **3-to-8** decoder has **3** inputs, **8** outputs; with an **enable** input decoders can be **cascaded**.

Memory Trick

- "Each decoder output = one minterm; OR the needed minterms to build any function."

CBT Trap

- A decoder output line corresponds to a **single minterm**; to build a function you OR the minterm lines where **F = 1** - using AND instead is the trap.

6.9 Magnitude Comparators

- A **magnitude comparator** compares two n-bit numbers and outputs three signals: **A>B**, **A=B**, **A<B**.
- Equality is detected by **XNOR** of each bit pair ANDed together: **A=B when all bit-XNORs are 1**.
- A 1-bit comparator: **A>B = A.B'**, **A<B = A'.B**, **A=B = A XNOR B**.

CBT Trap

- **A=B** uses **XNOR** per bit (equality), not XOR; greater/less-than logic starts from the **MSB** downward.

6.10 Code Converters (e.g., Binary-to-Gray)

- A **code converter** is a combinational circuit translating one code to another (BCD-to-Excess-3, Binary-to-Gray, etc.).
- Binary-to-Gray uses **XOR of adjacent bits**: **G_i = B_i XOR B_(i+1)**, with **G_MSB = B_MSB**.
- Gray-to-Binary uses **cumulative XOR**: **B_i = G_i XOR B_(i+1)**.

CBT Trap

- Binary-to-Gray is a **single XOR per bit**; Gray-to-Binary is a **chained/cumulative XOR** - direction confusion is the trap.

6.11 Parity Generator/Checker

- A **parity generator** produces the parity bit using a **cascade of XOR** gates over the data bits.
- A **parity checker** XORs all received bits (data + parity); output **1** signals an **error** (for even parity).
- Even-parity bit = **XOR of all data bits**; odd-parity bit = **XNOR / complement of that XOR**.

CBT Trap

- **Even-parity** generator = plain XOR tree; **odd-parity** = XNOR (one extra inversion). A checker detecting **odd** numbers of errors only - cannot find **even-count** errors.

7. Latches and Flip-Flops

7.1 Latches: SR Latch, Gated SR, D Latch

- A **latch** is a **level-sensitive** bistable memory element; it changes state whenever inputs change while **enabled** (no clock edge needed).
- An **SR latch** (NOR-based) has Set and Reset inputs: **S=1,R=0 -> Q=1**; **S=0,R=1 -> Q=0**; **S=R=0 -> hold**; **S=R=1 -> invalid**.
- A **NAND-based SR latch** is active-**low**; its forbidden condition is **S=R=0**.
- A **gated (clocked) SR latch** adds an **enable** so it responds only when enable = **1**.
- A **D latch** eliminates the invalid state by tying **D and its complement** to S and R: **Q follows D** while enabled (transparent).

Latch	Hold condition	Invalid condition
NOR SR	**S=R=0**	**S=R=1**
NAND SR	**S=R=1**	**S=R=0**
D latch	n/a	**none**

CBT Trap

- The **forbidden state** flips between NOR-SR (**S=R=1**) and NAND-SR (**S=R=0**); the **D latch** removes the invalid state entirely.

7.2 Flip-Flops: SR, D, JK, T

- A **flip-flop** is an **edge-triggered** bistable element; it changes only at a clock **edge**.
- **SR flip-flop**: like SR latch but clocked; **S=R=1** remains invalid.
- **D flip-flop**: **Q(next) = D**; stores one bit, used in registers.
- **JK flip-flop**: removes the invalid state; **J=K=1 -> toggle**; **J=K=0 -> hold**.
- **T (toggle) flip-flop**: **T=1 -> toggle**, **T=0 -> hold**; obtained by tying J=K=T.

FF	Next state Q(t+1)	Special
SR	**S + R'Q**	**S=R=1 invalid**
D	**D**	no invalid
JK	**JQ' + K'Q**	**J=K=1 toggles**
T	**T XOR Q**	toggle when T=1

Memory Trick

- "D = Data/Delay, T = Toggle, JK = no-forbidden SR, SR = Set/Reset with a forbidden 11."

CBT Trap

- **JK** resolves the SR **invalid (11)** state by **toggling**; the **D** flip-flop is essentially a **delay** (Q(t+1)=D).

7.3 Characteristic Tables and Excitation Tables

- A **characteristic table** gives **Q(t+1)** from inputs and **Q(t)**; the **characteristic equation** is its algebraic form.
- Characteristic equations: **D: Q+ = D**; **T: Q+ = T XOR Q**; **JK: Q+ = JQ' + K'Q**; **SR: Q+ = S + R'Q** (with SR=0).
- An **excitation table** gives the **required inputs** to achieve a desired state transition; it is the **reverse** of the characteristic table and is used in **sequential design**.

Transition Q->Q+	SR	JK	D	T
0 -> 0	0 X	0 X	0	0
0 -> 1	1 0	1 X	1	1
1 -> 0	0 1	X 1	0	1
1 -> 1	X 0	X 0	1	0

CBT Trap

- The **JK excitation** has the most **don't-cares (X)**, making it flexible in design; memorize **T toggles on 0->1 and 1->0** (T=1 for any change).

7.4 Flip-Flop Conversions (e.g., JK to D)

- Conversion procedure: write the **characteristic table** of the **desired** FF, use the **excitation table** of the **available** FF, then K-map the available FF's inputs.
- **JK to D**: **J = D**, **K = D'**.
- **D to JK**: **D = JQ' + K'Q**.
- **JK to T**: **J = T**, **K = T** (tie together).
- **D to T**: **D = T XOR Q**; **T to D**: **T = D XOR Q**.

Memory Trick

- "JK->D: J=D, K=D-bar"; "JK->T: tie J=K=T".

CBT Trap

- For JK->D, **K = D'** (complement), not D; forgetting the inversion is the standard mistake.

7.5 Edge-Triggered vs Level-Triggered; Master-Slave

- **Level-triggered (latch)**: responds during the entire active **level** of the clock (transparent) - prone to **multiple changes**.
- **Edge-triggered (flip-flop)**: responds only at the **rising** or **falling** clock edge.
- A **master-slave** flip-flop cascades **two** latches on **opposite** clock phases so the output updates **once per clock period**, behaving edge-like.

Type	Triggers on	Transparency
Level/Latch	**clock level**	**transparent**
Edge FF	**clock edge**	**not transparent**
Master-slave	**two phases**	**updates once/cycle**

CBT Trap

- A **latch is level-triggered**, a **flip-flop is edge-triggered**; master-slave uses **two latches** to mimic edge behaviour and avoid race-around.

7.6 Setup and Hold Time; Race-Around Condition

- **Setup time (t_su)**: minimum interval the data must be **stable BEFORE** the clock edge.
- **Hold time (t_h)**: minimum interval the data must remain **stable AFTER** the clock edge.
- Violating setup/hold causes **metastability** (output settles unpredictably).
- The **race-around condition** afflicts a **level-triggered JK with J=K=1**: the output **toggles repeatedly** while the clock is high if the pulse width > propagation delay.
- Cures: make **clock pulse < propagation delay**, or use a **master-slave** / **edge-triggered** JK.

Memory Trick

- "Setup is Before, Hold is after - S before, H after the edge."

CBT Trap

- **Race-around** occurs only with a **level-triggered JK at J=K=1**; the fix is **master-slave / edge-triggering**. Setup is **before** the edge, hold is **after** - swapping them is the trap.

8. Registers and Counters

8.1 Registers; Shift Registers (SISO, SIPO, PISO, PIPO)

- A **register** is a group of **n** flip-flops storing an **n-bit** word; loading all bits together is a **parallel** register.
- A **shift register** moves data one position per clock; classified by how data enters/leaves:
- **SISO** = Serial-In Serial-Out; **SIPO** = Serial-In Parallel-Out; **PISO** = Parallel-In Serial-Out; **PIPO** = Parallel-In Parallel-Out.
- An n-bit SISO register needs **n** clock pulses to load and **n** to read out serially.

Type	Input	Output	Use
SISO	serial	serial	**delay line**
SIPO	serial	parallel	**serial-to-parallel**
PISO	parallel	serial	**parallel-to-serial**
PIPO	parallel	parallel	**temporary store**

CBT Trap

- SIPO does **serial-to-parallel** conversion; PISO does **parallel-to-serial** - matching the conversion direction to the name is the trap.

8.2 Universal Shift Register (Concept)

- A **universal shift register** can **shift left**, **shift right**, **parallel load**, and **hold** - all four operations, selected by **2** mode-control lines.
- It is built from D flip-flops with a **4:1 MUX** at each stage choosing the operation.

CBT Trap

- A universal shift register supports **4** operations chosen by **2** select lines (shift-L, shift-R, load, hold) - fewer operations means it is not "universal".

8.3 Asynchronous (Ripple) Counters

- In an **asynchronous (ripple)** counter, only the first flip-flop sees the clock; each subsequent FF is clocked by the **previous FF's output**.
- Clock **ripples** through stages, so the total delay accumulates: worst-case delay **= n x t_pd** (proportional to number of FFs).
- A ripple counter of **n** flip-flops counts modulo **2^n**.
- Slow and prone to **decoding glitches**, but uses **minimal hardware**.

CBT Trap

- Ripple counters are **slow** (cumulative delay **n x t_pd**) and only one FF is driven by the **external clock**; do not call them synchronous.

8.4 Synchronous Counters

- In a **synchronous** counter, **all** flip-flops share the **same clock**, so they change **simultaneously**.
- Delay is just **one** flip-flop delay (plus gate delay), independent of n - much **faster** than ripple.
- Requires more **combinational logic** (steering gates) to decide which FFs toggle.

Feature	Ripple (async)	Synchronous
Clocking	**cascaded**	**common clock**
Speed	**slow (n.t_pd)**	**fast (1 t_pd)**
Hardware	**minimal**	**more gates**
Glitches	**more**	**fewer**

CBT Trap

- **Synchronous** = all FFs on one clock, fast, more logic; **asynchronous** = ripple, slow, less logic - the speed/hardware tradeoff is a standard MCQ.

8.5 Up/Down Counters; Mod-N Counters

- An **up counter** counts **0->max**; a **down counter** counts **max->0**; an **up/down** counter has a **direction control** line.
- A **mod-N counter** cycles through **N** distinct states (0 to N-1); it needs **ceil(log2 N)** flip-flops.
- To build a mod-N from a 2^n counter, **reset** the count at N (e.g., mod-10 resets at **1010** using a NAND-decoded clear).
- A **decade counter** is a **mod-10** counter (counts **0-9**).

Memory Trick

- "Mod-N needs ceil(log2 N) flip-flops"; mod-10 = decade needs **4** FFs.

CBT Trap

- A mod-N counter needs **ceil(log2 N)** flip-flops, e.g., mod-10 needs **4** (not 3); the reset is forced at count **N**, discarding states **N..2^n-1**.

8.6 Ring Counter and Johnson Counter

- A **ring counter** circulates a single **1** through n flip-flops; it has **n** states and **wastes** states (only n of 2^n used).
- A **Johnson (twisted-ring/switch-tail)** counter feeds back the **complemented** output; it has **2n** states.
- Ring counter needs **no decoding** gates (each state is one FF = 1); Johnson needs **minimal 2-input** decoding.

Counter	States with n FFs	Feedback
Ring	**n**	**Q of last**
Johnson	**2n**	**Q' of last**

Memory Trick

- "Ring = n states; Johnson = 2n states (Johnson is 'doubled' because of the twist)."

CBT Trap

- Ring counter = **n** states, Johnson = **2n** states - the Johnson's complemented feedback doubles the count; mixing the two state counts is the classic trap.

8.7 Counter Design from State Diagram (Concept)

- Synchronous counter design steps: **(1)** draw state diagram, **(2)** build state/transition table, **(3)** pick FF type and use its **excitation table**, **(4)** K-map each FF input, **(5)** draw circuit.
- Number of flip-flops = **ceil(log2 (number of states))**.
- **Unused states** should be checked to ensure the counter is **self-starting** (recovers into the main sequence).

CBT Trap

- Always verify **unused states** lead back into the cycle (**self-starting**); ignoring them can leave the counter **stuck/lock-up**.

9. Finite State Machines (Sequential Design)

9.1 Mealy Machine

- A **Mealy machine** output depends on **both present state and present inputs**.
- Outputs can change **asynchronously** with inputs (mid-state), often needing **fewer states** than Moore.
- Output is associated with **transitions (edges)** in the state diagram.

9.2 Moore Machine

- A **Moore machine** output depends **only on the present state**.
- Outputs are **synchronous** (stable through the state), associated with **states (nodes)**.
- A Moore machine usually needs **more states** than an equivalent Mealy machine but is **glitch-free**.

Feature	Mealy	Moore
Output depends on	**state + input**	**state only**
Output labeled on	**transitions**	**states**
State count	**fewer**	**more**
Output timing	**immediate**	**delayed one cycle**

Memory Trick

- "Mealy = inputs Matter to output; Moore = output Mapped to state only."

CBT Trap

- **Mealy** reacts **immediately** to inputs (fewer states); **Moore** reacts **one clock later** (more states, glitch-free) - the output-dependency definition is the guaranteed MCQ.

9.3 State Diagram and State Table

- A **state diagram** is a directed graph: nodes are **states**, edges are **transitions** labeled with input/output.
- A **state table** tabulates **present state, input -> next state, output**.
- Together they fully specify a **sequential circuit's behaviour**.

9.4 State Reduction/Minimization (Concept)

- **State reduction** merges **equivalent states** (states giving the same output and same next-state behaviour for all inputs) to reduce flip-flop count.
- Fewer states can mean fewer flip-flops (**ceil(log2 states)**), lowering cost.
- Tools: **implication chart** / **partitioning** method.

CBT Trap

- Two states are **equivalent** only if they match in output **AND** next-state for **every** input; reducing states may or may not reduce **flip-flops** (depends on crossing a power-of-2 boundary).

9.5 State Assignment (Concept)

- **State assignment** maps each symbolic state to a unique **binary code**.
- Choices like **binary**, **Gray**, or **one-hot** encoding affect the **combinational logic** complexity and speed.
- **One-hot** uses **one flip-flop per state** (n FFs for n states) - more FFs but **simpler/faster** next-state logic.

Encoding	FFs for N states	Logic
Binary	**ceil(log2 N)**	more complex
Gray	**ceil(log2 N)**	fewer glitches
One-hot	**N**	**simplest/fastest**

CBT Trap

- **One-hot** uses **N** flip-flops (one per state), trading more FFs for **faster** decode logic - it does NOT minimize flip-flop count.

10. Programmable & Memory Devices (Awareness)

10.1 ROM as Combinational Logic

- A **ROM** (Read-Only Memory) stores a fixed truth table; with **n** address lines it has **2^n** words and acts as a full **decoder + OR** array.
- A ROM realizes **any** combinational function of its address inputs because it stores **all 2^n** minterms.
- ROM structure = **fixed AND array (decoder) + programmable OR array**.

10.2 PLA and PAL (Concept)

- A **PLA (Programmable Logic Array)** has **both** the AND array and OR array **programmable** - most flexible.
- A **PAL (Programmable Array Logic)** has a **programmable AND** array but a **fixed OR** array - cheaper, faster, less flexible.
- A **ROM** has a **fixed AND** array (decoder) and **programmable OR** array.

Device	AND array	OR array
ROM	**fixed**	**programmable**
PAL	**programmable**	**fixed**
PLA	**programmable**	**programmable**

Memory Trick

- "PLA = both programmable; PAL = AND programmable; ROM = OR programmable."

CBT Trap

- **PLA** = both arrays programmable (flexible); **PAL** = only AND programmable; **ROM** = only OR programmable - the array-programmability grid is a frequent CIL MCQ.

10.3 Multiplexer/Decoder-Based Implementation (Revisited)

- A **2^n:1 MUX** implements any n-variable function by placing the **truth-table outputs** on its data inputs.
- A **decoder + OR** implements any SOP by ORing the **minterm** output lines.
- These are the **MSI** alternatives to gate-level design.

CBT Trap

- MUX-based: put **function outputs** on data lines; decoder-based: **OR the minterms** - mixing the two implementation styles is the trap.

11. Fixed and Floating Point

11.1 Fixed-Point Representation and Range

- **Fixed-point** representation places the **binary point** at a fixed position; integer and fraction fields have constant widths.
- For an unsigned format with i integer bits and f fraction bits: range **0** to **2^i - 2^(-f)**, resolution **2^(-f)**.
- Fixed-point is **fast and simple** but has **limited dynamic range** compared to floating-point.

CBT Trap

- Fixed-point resolution (smallest step) is **2^(-f)**; its narrow **dynamic range** is why floating-point exists.

11.2 Floating-Point Representation; Normalization

- **Floating-point** stores a number as **(-1)^sign x mantissa x base^exponent**, giving a huge **dynamic range**.
- The three fields are **sign**, **exponent (with bias)**, and **mantissa/significand**.
- **Normalization** adjusts the mantissa to the form **1.xxxx** (a single leading nonzero digit); the leading **1** is **implicit/hidden** in IEEE-754.
- The exponent is stored in **biased (excess)** form so it can be compared as an unsigned number.

CBT Trap

- The normalized leading **1** is **hidden** (not stored) in IEEE-754, giving one extra mantissa bit of precision for free.

11.3 IEEE 754: Single and Double Precision (Fields, Bias)

- **IEEE 754 single precision** = **32 bits**: **1** sign + **8** exponent + **23** mantissa; bias = **127**.
- **IEEE 754 double precision** = **64 bits**: **1** sign + **11** exponent + **52** mantissa; bias = **1023**.
- The stored **biased exponent** = **actual exponent + bias**; actual = **stored - bias**.
- Value (normalized) = **(-1)^S x 1.M x 2^(E - bias)**.
- With the hidden bit, single precision gives **24** bits of effective mantissa, double gives **53**.

Format	Total bits	Sign	Exponent	Mantissa	Bias
Single	**32**	**1**	**8**	**23**	**127**
Double	**64**	**1**	**11**	**52**	**1023**

Memory Trick

- Single: "**1-8-23**, bias **127**"; Double: "**1-11-52**, bias **1023**". Bias = **2^(e-1) - 1**.

CBT Trap

- Bias = **2^(exp_bits-1) - 1** = **127** (single) / **1023** (double); do not use 128 or 1024. Mantissa effective bits include the **hidden 1** (24 / 53).

11.4 Overflow and Underflow

- **Overflow** occurs when the exponent exceeds the maximum representable value -> result becomes **infinity**.
- **Underflow** occurs when the exponent is too negative (smaller than minimum) -> result approaches **zero** / becomes **denormal**.
- Special IEEE values: exponent all **1s** with zero mantissa = **infinity**; all **1s** with nonzero mantissa = **NaN**; exponent all **0s** = **zero or denormal**.

Exponent	Mantissa	Meaning
all 0s	0	**+/- zero**
all 0s	nonzero	**denormal**
all 1s	0	**+/- infinity**
all 1s	nonzero	**NaN**

CBT Trap

- **Infinity** = exponent all 1s with mantissa **0**; **NaN** = exponent all 1s with mantissa **nonzero** - the mantissa distinguishes them.

11.5 Floating-Point Addition/Multiplication (Concept)

- Floating-point **addition** steps: **(1)** align exponents (shift the smaller mantissa right), **(2)** add mantissas, **(3)** normalize, **(4)** round.
- Floating-point **multiplication**: **add the exponents (subtract one bias)**, **multiply mantissas**, normalize, round.
- **Rounding** modes include **round-to-nearest-even (default)**, toward zero, toward +/- infinity.

Memory Trick

- "Add FP: align then add; Multiply FP: multiply mantissas, add exponents (minus one bias)."

CBT Trap

- In FP multiplication you **add** exponents and **subtract the bias once** (because both stored exponents include the bias); forgetting to remove the extra bias is the trap.

12. Exam Focus / Practice Checklist

12.1 Base Conversions and Complement Arithmetic

- Decimal->base uses **repeated division (integer)** and **repeated multiplication (fraction)**; **2's complement** = invert + **1**.
- A->B subtraction = **A + 2's complement of B**, **discard** the final carry.

12.2 K-map Minimization (SOP/POS, with don't-cares)

- Group **1s** for SOP, **0s** for POS; use **don't-cares** freely to enlarge groups; a **2^k** group drops **k** variables.

12.3 Counting Prime / Essential Prime Implicants

- A **PI** cannot be enlarged; an **EPI** covers a minterm no other PI covers; the minimal cover = **all EPIs** + minimal extra PIs.

12.4 NAND/NOR Universal Realization

- **NAND** and **NOR** are universal; NOT = **single self-tied gate**; SOP -> **NAND-NAND**, POS -> **NOR-NOR**.

12.5 MUX/Decoder-Based Function Implementation

- **2^n:1 MUX** (n selects) or **n-to-2^n decoder + OR** implements any n-variable function; MUX takes **function outputs** on data lines.

12.6 Flip-Flop Excitation and Conversion

- Memorize excitation tables; **JK->D: J=D, K=D'**; **JK->T: J=K=T**; characteristic eqns **D:Q+=D**, **T:Q+=T XOR Q**, **JK:Q+=JQ'+K'Q**.

12.7 Counter Mod/State-Sequence Questions

- Mod-N needs **ceil(log2 N)** FFs; ring = **n** states, Johnson = **2n** states; ripple delay = **n x t_pd**.

12.8 IEEE 754 Field/Bias Computation

- Single **1-8-23, bias 127**; double **1-11-52, bias 1023**; value = **(-1)^S x 1.M x 2^(E-bias)**.

12.9 Number Range for n-bit Representations

- Unsigned: **0 to 2^n - 1**; signed 2's comp: **-2^(n-1) to +2^(n-1) - 1**; total values always **2^n**.

Final Rapid-Recall Table

Topic	Must-know number/formula
2's complement	**invert + 1**
Overflow	**Cin XOR Cout at MSB**
Mod-N counter	**ceil(log2 N)** FFs
Ring vs Johnson	**n** vs **2n** states
MUX selects	**log2(inputs)**
Full adder carry	**AB+BCin+ACin**
IEEE single bias	**127**

Topic	Must-know number/formula
IEEE double bias	**1023**
Gray code change	**1 bit** per step
K-map 2^k group	drops **k** variables

CBT Trap (Master List)

- Confusing **carry-out** with **overflow**; **8421 BCD** not self-complementing while **2421** is; **duality** (keeps variables) vs **De Morgan** (negates variables).
- Mixing **minterm/maxterm** complement conventions; grouping **1s vs 0s** for SOP/POS; **ring (n)** vs **Johnson (2n)** states; **bias 127/1023** not 128/1024.